Lecture 1. Computer Arithmetic

Integers, floating-point numbers, rounding, machine epsilon, error analysis, cancellation

Contents

1.1 Storage units and characters **2**

1.2 Integers: fixed-point numbers **2**

1.3 Floating-point numbers **2**

1.4 Rounding and machine epsilon **3**

1.5 Floating-point arithmetic and its pitfalls **4**

1.1 Storage units and characters

A **bit** (binary digit, a word coined by Tukey) takes the values 0 and 1; a **byte** is 8 bits. Decimal prefixes (KB = 10^3, MB = 10^6, GB = 10^9, TB = 10^{12} bytes, ...) are distinguished from binary ones (KiB = 2^{10}, MiB = 2^{20} bytes, ...).

Text is stored as character codes. ASCII uses 7 bits (128 characters) and extended ASCII 8 bits (256). Unicode encodings (UTF-8, UTF-16, UTF-32) cover all scripts; UTF-8, whose one-byte codes agree with ASCII, is dominant on the internet.

The basic lesson of this lecture: memory is finite, so *every* representation of $\mathbb{Z}$ or $\mathbb{R}$ in a computer is an approximation, and its imperfections must be understood.

1.2 Integers: fixed-point numbers

A fixed-point system with M bits models integers.

- **Unsigned** integers represent $\{0, 1, \ldots, 2^M - 1\}$ (e.g. 0–255 for 8 bits).
- **Signed** integers use the most significant bit as a sign bit and **two's complement** for negatives: to represent $-x$, flip all bits of x and add 1. Equivalently, the bit pattern of $-x$ is that of the unsigned integer $2^M - x$. The range is $[-2^{M-1}, 2^{M-1} - 1]$; for $M = 32$ the maximum is $2^{31} - 1 = 2147483647$.

Two's complement makes subtraction a special case of addition: adding the bit patterns modulo 2^M gives the right answer whenever the true result is in range. For example, with 8 bits, $57 - 35$: $57 = 00111001_2$ and $-35 = 11011101_2$, and the sum modulo 2^8 is $00010110_2 = 22$.

When the result leaves the range we get **overflow**. Languages differ: C and Julia wrap around (modular arithmetic, so $255 + 1 = 0$ for unsigned 8-bit integers), while R returns `NA`. Arbitrary-precision ("big") integers avoid overflow at the cost of speed.

1.3 Floating-point numbers

A real number is written in scientific notation with base $b = 2$:

$$\pm d_1.d_2 \cdots d_p \times 2^e, \qquad d_i \in \{0, 1\}.$$

In **normalized** form $d_1 = 1$ (e.g. $18 = 1.0010_2 \times 2^4$), so the leading bit need not be stored. A number that can be represented exactly is a **floating-point number**; every other real number is represented by a nearby floating-point number. The IEEE 754 standard (1985) stores

- a sign bit,
- the fraction (mantissa, significand) of the normalized representation, $p - 1$ bits,
- the exponent plus a **bias**.

format	bits	fraction bits	p	$e_{\min}, e_{\max}$	range	decimal digits
half (Float16)	16	10	11	$-14, 15$	$10^{\pm 4}$	3.3
single (float)	32	23	24	$-126, 127$	$10^{\pm 38}$	7.2
double	64	52	53	$-1022, 1023$	$10^{\pm 308}$	15.9

The bias is 15, 127 and 1023 respectively; the precision in decimal digits is $\log_{10} 2^p$. For half precision, the value of the bit string $b_{15}b_{14}\cdots b_0$ is

$$(-1)^{b_{15}} \times 2^{(b_{14}\cdots b_{10})_2 - 15} \times \Big(1 + \sum_{i=1}^{10} b_{10-i} 2^{-i}\Big).$$

Double precision is the default for numerical computing (R has no native single precision).

Special values. The exponent codes $e_{\min} - 1$ (all zeros) and $e_{\max} + 1$ (all ones) are reserved.

- Exponent $e_{\max} + 1$ with zero fraction: $\pm\infty$ (e.g. $10^{300} \times 10^{300}$).
- Exponent $e_{\max}+1$ with nonzero fraction: `NaN` (e.g. $0/0$, $0\cdot\infty$). A `NaN` is not equal to anything, including itself.
- Exponent $e_{\min} - 1$ with zero fraction: an exact zero (which cannot be written in normalized form).
- Exponent $e_{\min} - 1$ with nonzero fraction: **denormalized** (subnormal) numbers $0.d_2 \cdots d_p \times 2^{e_{\min}}$, filling the gap $(0, 2^{e_{\min}})$. This **gradual underflow** guarantees that $x - y = 0$ only if $x = y$. For example, in single precision $2^{-149} = 0.0\cdots01_2 \times 2^{-126}$ is representable.

1.4 Rounding and machine epsilon

A real number with more than p significant bits must be rounded. IEEE 754's default is **round to nearest, ties to even**: choose the nearest representable number, and if the number lies exactly midway, choose the one whose last bit is 0. With $p = 4$:

exact	rounded	reason
1.000011×2^1	1.000×2^1	remainder $011 < 100$: down
1.000110×2^1	1.001×2^1	remainder $110 > 100$: up
1.011100×2^1	1.100×2^1	tie: to even (up)
1.010100×2^1	1.010×2^1	tie: to even (down)

If x is represented by $[x]$, the **absolute error** is $|[x] - x|$ and the **relative error** is $|[x] - x|/|x|$.

Floating-point numbers are not evenly spaced: each interval $(2^i, 2^{i+1}]$ contains the same number (2^{p-1}) of them, uniformly spaced, so the spacing doubles from one binade to the next.

Definition (Machine epsilons)

$\epsilon_{\max}$ is the smallest positive floating-point number whose addition to 1 gives a result different from 1; $\epsilon_{\min}$ is the smallest positive floating-point number whose subtraction from 1 gives a result different from 1.

Proposition 1.1

With precision p and ties to even,

$$\epsilon_{\max} = 2^{-p} + 2^{-(2p-1)}, \qquad \epsilon_{\min} = 2^{-(p+1)} + 2^{-2p}.$$

For double precision, $\epsilon_{\max} = 2^{-53} + 2^{-105} \approx 1.1 \times 10^{-16}$.

Proof. The floating-point neighbours of 1 are $1 + 2^{-(p-1)}$ above and $1 - 2^{-p}$ below. The real number $1 + 2^{-p}$ is exactly midway between 1 and $1 + 2^{-(p-1)}$, and the tie goes to the even neighbour 1.

So $\epsilon_{\max} > 2^{-p}$. The next floating-point number after $2^{-p} = 1.0\cdots0_2 \times 2^{-p}$ is $1.0\cdots01_2 \times 2^{-p} = 2^{-p} + 2^{-p-(p-1)}$, and adding it to 1 gives a number past the midpoint, which rounds to $1 + 2^{-(p-1)}$. Hence $\epsilon_{\max} = 2^{-p} + 2^{-(2p-1)}$. Similarly, $1 - 2^{-(p+1)}$ is midway between $1 - 2^{-p}$ and 1 and rounds to the even value 1; the next floating-point number after $2^{-(p+1)}$ is $2^{-(p+1)} + 2^{-(p+1)-(p-1)} = 2^{-(p+1)} + 2^{-2p}$, which does move 1. ■

Theorem 1.2 (Relative error of representation)

If a real number $x > 0$ in the normal range is represented by $[x]$, then

$$\frac{|[x] - x|}{|x|} \le \epsilon_{\max}.$$

Proof. Let $[x] = (1 + \sum_{i=1}^{p-1} b_{i+1} 2^{-i}) \times 2^e$, so $[x] \ge 2^e$ and the spacing of floating-point numbers near $[x]$ is $2^{e-(p-1)}$. Rounding to nearest gives $|x - [x]| \le 2^e/2^p$. Then

$$\frac{|x - [x]|}{|x|} \le \frac{2^e}{2^p} \cdot \frac{1}{[x] - 2^e/2^p} \le \frac{2^e}{2^p} \cdot \frac{1}{2^e(1 - 2^{-p})} \le \frac{1}{2^p}\Big(1 + \frac{1}{2^{p-1}}\Big) = \epsilon_{\max},$$

using $1/(1-u) \le 1 + 2u$ for $0 \le u \le \frac{1}{2}$. ■

So $[x] = x(1+\delta)$ with $|\delta| \le \epsilon_{\max}$: the machine epsilon is the **unit roundoff** or machine precision.

Example 1.1 (A toy decimal system)

Take $b = 10$, $p = 5$, $-9 \le e \le 9$, numbers $\pm 0.d_1 \cdots d_5 \times 10^e$, normalized except that subnormals and zero are allowed at $e = -9$. Let $X \sim \mathrm{Unif}(0, 1)$ and round to nearest.

- The smallest positive number is $0.00001 \times 10^{-9} = 10^{-14}$, so $[X] = 0$ iff $X \le \frac{1}{2} \cdot 10^{-14}$ (ties to the even neighbour 0): $P([X] = 0) = 5 \times 10^{-15}$.
- $1 = 0.10000 \times 10^1$ and its predecessor is 0.99999×10^0, so $[X] = 1$ iff $X \ge 0.999995$: $P([X] = 1) = 5 \times 10^{-6}$.
- $[X]$ is not uniform over the floating-point numbers in $(0, 1)$: within $[10^{e-1}, 10^e)$ the spacing is 10^{e-5}, so each number there has probability 10^{e-5}. For instance $P([X] = 0.5) = 10^{-5}$ but $P([X] = 0.05) = 10^{-6}$.

1.5 Floating-point arithmetic and its pitfalls

IEEE 754 requires that $+, -, \times, \div, \sqrt{\ }$, remainder and conversions be computed *as if exactly* and then rounded: the computed sum of floating-point numbers $[x]$ and $[y]$ is $[[x] + [y]]$.

Overflow and underflow. Results beyond $\approx 10^{308}$ in magnitude overflow to $\pm\infty$; results below $\approx 10^{-308}$ underflow gradually to subnormals and then 0. Underflow is usually the lesser evil, so formulas should be arranged to avoid overflow. For instance, the logistic function should be evaluated as

$$p = \frac{1}{1 + e^{-x^\top \beta}} \qquad \text{rather than} \qquad \frac{e^{x^\top \beta}}{1 + e^{x^\top \beta}},$$

since the latter gives $\infty/\infty =$ `NaN` for large $x^\top\beta$, while the former only underflows gracefully. Similarly $\log(e^{709})$ is fine but $\log(e^{710}) = \infty$; computations with likelihoods should be done on the log scale.

Adding numbers of different magnitudes. If $a = x.xx\cdots \times 2^{30}$ and $b = y.yy\cdots \times 2^{-30}$, then b falls below the last bit of a and $a + b = a$ in double precision: the information in b is lost.

Proposition 1.3 (Error of a sum)

For $x, y > 0$, the computed sum satisfies

$$\frac{|[[x] + [y]] - (x + y)|}{x + y} \leq 2\epsilon_{\max} + \epsilon_{\max}^2 \approx 2\epsilon_{\max}.$$

Proof. Write $[x] = x(1+\delta_x)$, $[y] = y(1+\delta_y)$ and $[[x]+[y]] = ([x]+[y])(1+\delta_s)$, all $|\delta| \leq \epsilon_{\max}$. Then $[x] + [y] \leq (x + y)(1 + \epsilon_{\max})$ and

$$|[[x] + [y]] - (x + y)| \leq |\delta_s|([x] + [y]) + |\delta_x|x + |\delta_y|y \leq \epsilon_{\max}(1 + \epsilon_{\max})(x + y) + \epsilon_{\max}(x + y).$$

■

So adding positive numbers is benign. Subtraction is not.

Catastrophic cancellation. Subtracting two nearly equal numbers that carry rounding errors destroys significant digits. With 3 decimal digits, $b = 3.34$, $a = 1.22$, $c = 2.28$: the exact discriminant is $b^2 - 4ac = 0.0292$, but $b^2 = 11.2$ and $4ac = 11.1$ after rounding, giving 0.1. The subtraction itself was exact; the errors were introduced earlier and then magnified. In binary: if $[x] = 1.d_2 \cdots d_{p-1}1$ and $[y] = 1.d_2 \cdots d_{p-1}0$, then $[x] - [y] = 2^{-(p-1)}$ exactly, while the true $x - y$ may be anywhere in $(0, 2^{-(p-2)} + 2^{-2p}]$. The relative error is unbounded: no significant digit is guaranteed. Likewise $1 - \Phi(z)$ computed as one minus the cdf is useless for large z (it returns 0 for $z \geq 9$), and tail probabilities must be computed directly.

Rules of thumb.

(1) Add small numbers before large ones.

(2) Add numbers of like magnitude together (pairwise summation).

(3) Avoid subtracting nearly equal numbers. For example, for $ax^2 + bx + c = 0$ with $b > 0$, compute $x_1 = \frac{-b-\sqrt{b^2-4ac}}{2a}$ (no cancellation) and then $x_2 = c/(ax_1)$ from $x_1x_2 = c/a$, instead of $\frac{-b+\sqrt{b^2-4ac}}{2a}$.

Floating-point arithmetic also violates familiar algebraic laws: addition is not associative ($(a+b)-a \neq b$ when $a \gg b$) and multiplication does not distribute over addition exactly. Even the additive identity misbehaves: zero has two representations ± 0 (equal under comparison), $x + 0 = x$ fails for $x =$ `NaN` (`NaN` equals nothing), and $x - x = 0$ fails for $x = \pm\infty$ and `NaN`. Familiar decimal constants are also inexact: 0.1, 0.3 have infinite binary expansions, so `0.1+0.2==0.3` and `0.1==0.3/3` are both false; equality of computed reals should be tested with a tolerance (R's `all.equal`).

Example 1.2 (Cancellation in the quadratic formula)

For $a = 10^{-30}$, $b = -10^{30}$, $c = 10^{30}$ the roots are about 10^{60} and 1. In double precision $4ac = 4$ is far below the last bit of $b^2 = 10^{60}$, so the computed discriminant equals b^2 and $\sqrt{b^2 - 4ac} = |b|$ exactly. The textbook formula then returns $(-b - |b|)/(2a) = 0$ for the small root: every digit is lost to cancellation. The stable recipe is

$$q = -\tfrac{1}{2}(b + \operatorname{sign}(b)\sqrt{b^2 - 4ac}), \qquad x_1 = q/a, \qquad x_2 = c/q,$$

which never subtracts nearly equal numbers and gives $x_2 = 1$ to full precision. (With $b = +10^{30}$ the same happens with the signs reversed.)

Example 1.3 (The harmonic series "converges")

Summing $1/i$ in a loop, the partial sums $s_n \approx \log n$ diverge mathematically, but on a computer they stop changing: once $1/n$ is below half the spacing of floating-point numbers near s_n, the addition $s_n + 1/n$ rounds back to s_n. In single precision this happens at $n = 2^{21} = 2097152$ with $s = 15.4037$; in double precision the spacing near $s \in [32, 64)$ is 2^{-47}, so the sum freezes near 34 after about $2^{48} \approx 3 \times 10^{14}$ terms.

The same series gives Euler's constant $\gamma = \lim_n(\sum_{i \le n} 1/i - \log n) \approx 0.5772$, i.e. $f(n) = \log n$ in $\sum_{i \le n} 1/i = f(n) + \gamma + o(1)$. To compute it accurately: choose a moderate n (say 10^6), add the terms in *increasing* order $1/n + \cdots + 1/1$ (small numbers first), subtract $\log n$, and remove the leading error term, which is $1/(2n)$. With $n = 10^6$ this gives γ to about 13 digits, while the uncorrected difference is off by 5×10^{-7}.